

Supplementary Material: Inferring Implicit Constraints from Service Composition Requests

Anonymous Submission

Abstract. This document provides supplementary material for the IC-SOC 2026 paper *Inferring Implicit Constraints from Service Composition Requests: A Framework and Evaluation of LLM Prompting Strategies*. It contains extended agreement metrics, extended evaluation results, annotation guidelines, prompt templates, the unified JSON schema, reproducibility instructions, and a disclosure of AI tool usage. All research artifacts (benchmark, prompts, evaluation scripts, and this document) are available at the anonymous repository linked in the paper.

1 Annotation Matching Configurations

Two distinct matching configurations were used, serving different purposes, and they must not be conflated.

IAA matching configuration (used in the human-model agreement analysis, Section 4.1 of the paper). Similarity threshold: 0.45. Category bonus: 0.10. Normalisation: aggressive (punctuation stripped, tokens lowercased and stemmed before comparison). This stricter configuration produces a conservative reliability estimate: surface wording differences introduced by the LLM annotation pass should not artificially inflate agreement scores. The higher threshold accepts only pairs where token overlap is strong before the category bonus is applied.

Evaluation matching configuration (used in the RQ2 prompting experiment, Section 4.3 of the paper). Similarity threshold: 0.25. Category bonus: 0.05. Normalisation: standard (lowercased tokens only). This more permissive configuration accommodates legitimate paraphrase variation in model outputs. Penalising a model for rephrasing a correctly identified constraint in different words would underestimate inference quality. The threshold is intentionally low; label correctness is assessed independently after alignment.

The two configurations are implemented as separate scripts to keep the distinction explicit.¹ The combined similarity score in both cases is $0.6 \times J(g, p) + 0.4 \times S(g, p)$, where J is the Jaccard coefficient over token sets and S is the character-level SequenceMatcher ratio; the only difference between configurations is the acceptance threshold, the category bonus weight, and the normalisation depth applied before scoring.

¹ `iaa/run_iaa_eval.py` and `RQ2/eval/evaluate_rq2.py`

Table 1. Full human–model agreement metrics by domain. The top block reproduces the metrics from the paper; the lower block reports the additional overlap metrics not included for space reasons.

Metric	Travel Healthcare	
	Travel	Healthcare
Requests compared	25	25
Exact count agreement	0.48	0.48
Density kappa	0.576	0.450
Category kappa (aligned pairs)	1.000	0.973
Resolvability kappa	0.742	0.936
Importance kappa	0.949	0.753
Avg overlap precision	0.609	0.731
Avg overlap recall	0.514	0.673
Avg overlap F1	0.545	0.688

2 Extended Agreement Metrics

Table 1 extends Table 2 of the paper with three additional metrics measuring pairwise overlap between annotation passes before label assignment. These were computed on aligned constraint pairs under the IAA matching configuration (Section 1).

The overlap precision, recall, and F1 measure how well the two passes agree on *which* constraints exist before considering how they label them. Healthcare shows higher overlap than travel, consistent with healthcare’s Logical-heavy profile, where constraints are more unambiguous to identify even when they are hard to label precisely. Lower overlap in travel concentrates at requests with multiple Domain-default constraints, where the LLM pass tended to aggregate what the human pass split into individual entries.

3 Extended Evaluation Results

3.1 Per-Category Precision and Recall

Table 2 reports precision and recall for each category by method and domain, complementing the F1 values in Table 6 of the paper. The pattern is consistent with the paper’s discussion: high recall on Temporal and Spatial with moderate-to-low precision indicates over-generation within those categories, while Domain-default shows low recall across most conditions, confirming that the category is systematically under-recovered.

3.2 Resolvability and Importance Label Accuracy

Table 3 reports labeling accuracy on the resolvability and importance dimensions, measured on matched constraint pairs only (i.e., pairs accepted by the

Table 2. Per-category precision (P) and recall (R) by method and domain. T = Travel, HC = Healthcare.

Dom.	Method	Temporal		Spatial		Logical		Dom.-def.	
		P	R	P	R	P	R	P	R
T	Zero-shot	0.483	0.558	0.571	0.800	0.250	0.286	0.050	0.379
	Few-shot	0.700	0.635	0.769	0.769	0.667	0.667	0.533	0.788
	CoT	0.600	0.738	0.556	0.714	0.231	0.364	0.000	0.000
	Hybrid	0.714	0.714	0.769	0.769	0.500	0.750	0.400	0.529
HC	Zero-shot	0.696	0.759	0.500	0.600	0.333	0.576	0.000	0.000
	Few-shot	0.783	0.783	0.750	0.750	0.444	0.727	0.000	0.000
	CoT	0.750	0.818	0.545	0.667	0.400	0.667	0.200	0.100
	Hybrid	0.783	0.783	0.889	0.889	0.615	0.727	0.000	0.000

Table 3. Resolvability and importance label accuracy on matched pairs. T = Travel, HC = Healthcare.

Dom.	Method	Resolvability acc.		Importance acc.	
		Travel	Healthcare	Travel	Healthcare
	Zero-shot	0.41	0.56	0.62	0.68
	Few-shot	0.68	0.71	0.79	0.81
	CoT	0.44	0.59	0.65	0.71
	Hybrid	0.61	0.74	0.73	0.79

evaluation matcher). The gap between category agreement (Table 6 of the paper) and resolvability accuracy is 20–30 percentage points across conditions, confirming that the resolvability dimension is harder to label than category regardless of prompting strategy. In-context examples close this gap more than the reasoning instruction does.

3.3 Density-Stratified Performance

Table 4 reports micro-F1 and micro-IC by request density level. Low-density requests (1–2 constraints) show the highest variance across methods, while High-density requests (5 or more constraints) consistently show lower recall, reflecting the compounding cost of missing constraints in constraint-rich requests. These results are directional rather than stable given the small per-slice counts (2–6 requests per cell).

4 Annotation Guidelines

This section reproduces the annotation protocol applied in both the manual pass and the LLM consistency pass. The framework definitions (categories, resolvability levels, importance levels) are provided in Section 2 of the paper. This

Table 4. Micro-F1 and micro-IC by density level, averaged across domains. Low: 1–2 constraints; Medium: 3–4; High: ≥ 5 .

Method	Metric	Low	Medium	High	All
Zero-shot	μ -F1	0.52	0.46	0.38	0.44
	μ -IC	0.63	0.55	0.44	0.54
Few-shot	μ -F1	0.76	0.69	0.61	0.67
	μ -IC	0.86	0.77	0.70	0.75
CoT	μ -F1	0.55	0.49	0.42	0.48
	μ -IC	0.65	0.58	0.47	0.57
Hybrid	μ -F1	0.73	0.68	0.60	0.65
	μ -IC	0.80	0.74	0.66	0.72

section documents the shared annotation principles, the explicit-vs-implicit decision rule, and the domain-specific resolution rules.

4.1 Shared Annotation Principles

Annotation objective. For each request, identify all planning-relevant implicit constraints and assign: constraint description, category, resolvability, and importance.

Inclusion rule. Only constraints that affect service composition, service selection, feasibility, or coordination are annotated. Information that is domain-internal and does not influence planning or composition decisions is excluded.

Exclusion rule. Explicitly stated constraints are not annotated unless additional inference or grounding is required for planning. A constraint directly specified in executable form is not annotated even if it is planning-relevant.

Counting rule. Each inferred constraint is counted individually. Composite expressions involving multiple distinct planning-relevant parameters produce multiple separate constraint entries.

Consistency rule. When the same expression recurs across requests, the same interpretation rule is applied consistently to preserve label comparability across the dataset.

4.2 Explicit vs. Implicit Decision Rule

A constraint is annotated only if inference is needed to recover planning-relevant information not directly specified in executable form. Table 5 summarises the boundary.

4.3 Travel Domain Resolution Rules

Temporal. Relative date expressions such as “tomorrow”, “next weekend”, or “by end of week” are resolved as Implicit and Critical. Event-anchored temporal

Table 5. Explicit vs. implicit constraint examples.

Expression	Annotate?	Reason
“wheelchair-accessible clinic”	No	Accessibility explicitly stated
“I use a wheelchair and cannot manage stairs”	Yes	Accessibility inferred from mobility limitation
“I have public insurance”	No	Insurance type explicitly stated
“avoid unexpected out-of-pocket costs”	Yes	Coverage compatibility inferred
“within 1 km”	No	Distance threshold explicitly stated
“near the conference venue”	Yes	Proximity inferred; threshold fuzzy
“Book a flight to Paris”	Yes	Passenger count = 1 inferred as domain-default

Table 6. Travel annotation consistency rules.

Expression type	Category	Resolvability	Importance
Proximity (near, walking distance)	Spatial	Vague	Useful
Relative future date (next week)	Temporal	Implicit	Critical
Time-of-day without fixed boundary	Temporal	Borderline	Useful
Price or cost sensitivity (cheap)	Logical	Vague	Useful
Domain-default (passenger/-room count)	Domain-default	Implicit	Critical

references are annotated only when they directly affect travel planning. Arrival or departure timing relative to an event is not inferred unless explicitly stated.

Spatial. Proximity expressions such as “near” and “walking distance” are resolved as Spatial, Vague, and Useful. Location grounding is applied when a landmark or district must be mapped to a city to identify candidate services (e.g., Rijksmuseum → Amsterdam). Grounding from a city to its country is not annotated.

Logical. Numerical thresholds, cost limits, and layover limits are resolved as Logical constraints when they affect service selection or feasibility. Optimization goals such as minimising price are resolved as Logical and Critical. Conditional dependencies are resolved as Logical planning constraints.

Domain-default. Default passenger count for a single-traveller flight booking is 1; default room and guest count for hotel bookings is 1. Minimum driver age for rental services is applied as a domain-default when no explicit age constraint is stated. Defaults are not annotated when the corresponding information is explicitly provided.

Table 7. Healthcare annotation consistency rules.

Expression type	Category	Resolvability	Importance
Proximity (near, central)	Spatial	Vague	Useful
Relative future date (next week)	Temporal	Implicit	Critical
Time-of-day without fixed boundary	Temporal	Borderline	Useful
Avoidance of additional costs	Logical	Implicit	Critical
Physical limitation implying accessibility	Logical	Implicit	Critical

4.4 Healthcare Domain Resolution Rules

Temporal. Relative date expressions referring to the immediate future are resolved as Implicit and Critical. Time-of-day expressions such as “morning” or “evening” are resolved as Borderline and Useful. Urgency expressions implying immediate care are resolved as Implicit and Critical. Relative timing references anchored to prior events (e.g., “two weeks after my surgery”) are resolved as Implicit and Critical when they impose a required temporal relationship.

Spatial. Proximity expressions such as “near” and “central” are resolved as Spatial, Vague, and Useful. References to named districts or administrative areas are resolved as Implicit and Critical when grounding is required to identify the applicable service region.

Provider capability inference. Physical or functional limitations implied by the request are resolved as Logical, Implicit, and Critical constraints on provider accessibility. Requests implying coordination of multiple services within a single visit are resolved as Logical, Implicit, and Critical service-bundling constraints. Expressions indicating absence of administrative steps (e.g., no referral requirement) are resolved as Logical, Implicit, and Critical, as they affect booking eligibility.

Insurance and payment. Expressions indicating avoidance of additional charges are resolved as Logical, Implicit, and Critical, reflecting coverage compatibility requirements. Willingness to accept higher costs for faster service is resolved as a Logical, Implicit, and Critical pathway compatibility constraint.

Medical context inference. Medical context is used only to the extent that it affects service routing or booking feasibility. Injury or condition descriptions that imply urgency and result in routing to urgent or emergency care are resolved as Domain-default, Implicit, and Critical. The following are *never* inferred from medical context alone: specific clinic types based on symptoms; treatment modalities not explicitly mentioned; clinical procedures or protocols; patient demographics or medical history unless explicitly stated.

Appointment coordination. Requests implying same-day coordination of multiple services are resolved as Logical, Implicit, and Critical. Sequencing requirements that establish order dependencies between services are resolved as Logical, Implicit, and Critical.

4.5 Annotation Scope Boundaries

Annotators must not introduce constraints beyond those reasonably inferable from the request text. The following are prohibited in both domains: invented information not present in the request; constraints that rely on personal medical history, user profiling, or unstated preferences; over-interpretation beyond planning-relevant domain inference. The scope boundary keeps the benchmark focused on executable intent recovery rather than speculative personalisation.

The gold annotation files are available in the anonymous repository:

- annotations/gold/travel_gold.json
- annotations/gold/healthcare_gold.json

5 Prompt Templates

This section provides the full prompt text for each of the four prompting strategies evaluated in RQ2. The placeholders {request_id} and {request_text} are substituted at runtime by the generation pipeline. Prompt files are available in the repository under RQ2/<method>/<method>_prompt.md.

5.1 Zero-Shot Prompt

```

You are a system for structured constraint extraction from
natural-language requests to support automated service
composition.

You are given a natural-language service request.
Your task is to extract and classify all implicit
constraints.

An implicit constraint is information that:
- is not explicitly specified in executable form in the
  request, and
- must be inferred to support planning, service selection,
  or
  coordination.

CONSTRAINT CATEGORIES

Each constraint must be assigned exactly one category:
- Temporal: time, dates, ordering, duration, or scheduling
- Spatial: location, proximity, or geographic relationships
- Logical: dependencies, conditions, coordination, or
  optimization
- Domain-default: standard assumptions required to complete
  a
  valid request

```

```

RESOLVABILITY
- Implicit: can be deterministically resolved using general
  knowledge (e.g., calendar or common conventions)
- Vague: inherently fuzzy or subjective
- Borderline: has approximate interpretations but no
  universally
  fixed boundary

IMPORTANCE
- Critical: required for feasibility or correctness
- Useful: improves quality but not strictly required
- Optional: minor preference with negligible impact

TASK
Extract ALL implicit constraints from this request:
Request ID: {request_id}
"{request_text}"

Return ONLY valid JSON in the following format:
{
  "request_id": "{request_id}",
  "constraints": [
    {
      "id": "{request_id}_C1",
      "description": "concise natural language constraint",
      "category": "Temporal|Spatial|Domain-default|Logical",
      "resolvability": "Implicit|Vague|Borderline",
      "importance": "Critical|Useful|Optional",
      "notes": "short explanation or reason"
    }
  ],
  "constraint_count": 1,
  "density": "Low|Medium|High"
}

RULES
- Include only constraints that affect feasibility,
  coordination,
  or planning quality
- Do not extract explicitly stated constraints
- Do not introduce assumptions not grounded in the request
- constraint_count must equal the number of extracted
  constraints
- density: Low = 1-2, Medium = 3-4, High = 5 or more
- Constraint IDs: {request_id}_C1, {request_id}_C2, ...
- Output JSON only. No text outside the JSON.

```

Listing 1.1. Zero-shot prompt template.

5.2 Chain-of-Thought Prompt

The CoT prompt shares the same header (system role, category definitions, resolvability and importance levels, output schema, and rules) as the zero-shot prompt above. The distinguishing element is the six-step internal reasoning instruction inserted before the output format block.

```
Apply the following internal reasoning process:
INTERNAL REASONING STEPS (do not include in final output)
1. Scan the request and identify all elements not directly
   specified in executable form that may require inference,
   grounding, or interpretation.
2. From these, filter only those that affect planning,
   feasibility, service selection, or coordination.
3. Decompose each selected element into a distinct
   constraint,
   ensuring each represents a single atomic requirement.
4. For each constraint assign:
   - the appropriate category
   - its resolvability level
   - its importance level
5. Review to ensure:
   - no explicitly stated constraints are included
   - no unsupported assumptions are introduced
   - constraint count and density are consistent
6. Perform a final completeness and consistency check:
   - no relevant implicit constraints were missed
   - constraints are not duplicated or overlapping
   - the set fully captures the planning requirements
```

Listing 1.2. CoT reasoning instruction (inserted after TASK, before output format).

5.3 Few-Shot Prompt

The few-shot prompt shares the same header as the zero-shot prompt. The four annotated examples below are inserted between the task instruction and the output format block.

```
EXAMPLES

Example 1
Request ID: HEALTH_005
Request: Schedule a post-op wound check in Hamburg two
        weeks after
my knee surgery on 10 June 2026.
{
  "request_id": "HEALTH_005",
  "constraints": [
    {
```

```

    "id": "HEALTH_005_C1",
    "description": "follow-up date derived as surgery
        date + 14 days",
    "category": "Temporal",
    "resolvability": "Implicit",
    "importance": "Critical",
    "notes": "Post-op timing is medically prescribed."
  },
  {
    "id": "HEALTH_005_C2",
    "description": "wound check must occur after the
        surgical procedure",
    "category": "Logical",
    "resolvability": "Implicit",
    "importance": "Critical",
    "notes": "Medical necessity; ordering constraint."
  }
],
"constraint_count": 2,
"density": "Low"
}

```

Example 2

Request ID: HEALTH_025

Request: Book a specialist eye exam in Hamburg for the week after next near Altona station. I need it coordinated with a vision certificate on the same day. I'd prefer the regular covered route unless paying extra is the only way to get a faster slot.

```

{
  "request_id": "HEALTH_025",
  "constraints": [
    {
      "id": "HEALTH_025_C1",
      "description": "week after next requires calendar
          grounding to
          the second upcoming Monday-Sunday interval",
      "category": "Temporal",
      "resolvability": "Implicit",
      "importance": "Critical",
      "notes": "Date grounding needed to find available
          slots."
    },
    {
      "id": "HEALTH_025_C2",
      "description": "near Altona station implies fuzzy
          proximity",
      "category": "Spatial",

```

```

    "resolvability": "Vague",
    "importance": "Useful",
    "notes": "Subjective convenience preference."
  },
  {
    "id": "HEALTH_025_C3",
    "description": "eye exam and vision certificate must
      be
      coordinated within the same day",
    "category": "Logical",
    "resolvability": "Implicit",
    "importance": "Critical",
    "notes": "Explicitly requested same-day coordination
      ."
  },
  {
    "id": "HEALTH_025_C4",
    "description": "provider must support both services
      within
      the same facility",
    "category": "Logical",
    "resolvability": "Implicit",
    "importance": "Critical",
    "notes": "Required for same-day coordination to work
      ."
  },
  {
    "id": "HEALTH_025_C5",
    "description": "service must allow switching between
      public
      insurance and private appointment pathways",
    "category": "Logical",
    "resolvability": "Implicit",
    "importance": "Useful",
    "notes": "Conditional preference; affects available
      options."
  }
],
"constraint_count": 5,
"density": "High"
}

```

Example 3

Request ID: TRAVEL_013

Request: Find the cheapest flight from Split to London this week.

```

{
  "request_id": "TRAVEL_013",
  "constraints": [
    {

```

```

    "id": "TRAVEL_013_C1",
    "description": "this week = remaining days of the
                    current
                    calendar week until Sunday",
    "category": "Temporal",
    "resolvability": "Implicit",
    "importance": "Critical",
    "notes": ""
  },
  {
    "id": "TRAVEL_013_C2",
    "description": "minimize price",
    "category": "Logical",
    "resolvability": "Implicit",
    "importance": "Critical",
    "notes": ""
  },
  {
    "id": "TRAVEL_013_C3",
    "description": "1 passenger",
    "category": "Domain-default",
    "resolvability": "Implicit",
    "importance": "Critical",
    "notes": ""
  }
],
"constraint_count": 3,
"density": "Medium"
}

```

Example 4

Request ID: TRAVEL_006

Request: I want to visit my parents in Karol Bagh over
Christmas,
need a hotel cheaper than 30 EUR per night.

```

{
  "request_id": "TRAVEL_006",
  "constraints": [
    {
      "id": "TRAVEL_006_C1",
      "description": "Christmas period = fuzzy holiday
                      dates",
      "category": "Temporal",
      "resolvability": "Vague",
      "importance": "Critical",
      "notes": ""
    },
    {
      "id": "TRAVEL_006_C2",
      "description": "Karol Bagh resolves to Delhi",

```

```

        "category": "Spatial",
        "resolvability": "Implicit",
        "importance": "Critical",
        "notes": ""
    },
    {
        "id": "TRAVEL_006_C3",
        "description": "Number of rooms: 1",
        "category": "Domain-default",
        "resolvability": "Implicit",
        "importance": "Critical",
        "notes": ""
    },
    {
        "id": "TRAVEL_006_C4",
        "description": "Number of guests: 1",
        "category": "Domain-default",
        "resolvability": "Implicit",
        "importance": "Critical",
        "notes": ""
    },
    {
        "id": "TRAVEL_006_C5",
        "description": "budget threshold applied during hotel
                        selection",
        "category": "Logical",
        "resolvability": "Implicit",
        "importance": "Critical",
        "notes": ""
    }
],
"constraint_count": 5,
"density": "High"
}

```

Listing 1.3. Few-shot examples (two healthcare, two travel).

5.4 Hybrid Prompt

The hybrid prompt combines the four annotated examples from the few-shot template (Listing 1.3) with the six-step internal reasoning instruction from the CoT template (Listing 1.2) in a single prompt. The examples appear after the task instruction, followed by the reasoning steps, followed by the output format and rules. All other elements (system role, category definitions, resolvability and importance levels) are identical to the other three templates.

6 JSON Annotation and Output Schema

All four prompting strategies and both annotation passes produce output in the unified JSON schema below. The `metadata` block is present in annotation files only. The `constraints` array contains one entry per identified implicit constraint.

```
{
  "metadata": {
    "annotator_name": "string",
    "annotation_date": "DD-MM-YYYY",
    "domain": "travel_booking | healthcare_booking"
  },
  "annotations": [
    {
      "request_id": "DOMAIN_NNN",
      "domain": "travel_booking | healthcare_booking",
      "request": "User request text",
      "constraints": [
        {
          "id": "DOMAIN_NNN_C1",
          "description": "Concise natural-language constraint",
          "category": "Temporal | Spatial | Logical | Domain-default",
          "resolvability": "Implicit | Vague | Borderline",
          "importance": "Critical | Useful | Optional",
          "notes": "Brief justification or empty string"
        }
      ],
      "constraint_count": 1,
      "density": "Low | Medium | High"
    }
  ]
}
```

Listing 1.4. Unified annotation and output schema.

Field constraints. `constraint_count` must equal the length of the `constraints` array. `density` is derived from `constraint_count`: Low for 1–2, Medium for 3–4, High for 5 or more. Constraint IDs follow the pattern `DOMAIN_NNN_C1`, `DOMAIN_NNN_C2`, and so on.

7 Reproducibility Instructions

7.1 Prerequisites

- Python 3.10 or later.
- An active OpenAI API key (required only for regenerating LLM outputs; not required to rerun evaluation and analysis on stored outputs).

7.2 Installation

```
pip install -r requirements.txt
```

Listing 1.5. Dependency installation.

```
openai>=1.0.0
scikit-learn>=1.3.0
```

Listing 1.6. requirements.txt.

7.3 Environment Setup

```
export OPENAI_API_KEY="YOUR_API_KEY_HERE"
```

Listing 1.7. API key setup (Linux/macOS).

```
setx OPENAI_API_KEY "YOUR_API_KEY_HERE"
```

Listing 1.8. API key setup (Windows PowerShell).

7.4 Running the Evaluation

All evaluation outputs are stored in the repository under `RQ2/eval/results`. The following commands reproduce them from the stored LLM outputs without requiring API access.

```
python RQ2/eval/evaluate_rq2.py \
  --domain all \
  --method all \
  --match-threshold 0.25 \
  --category-bonus 0.05
```

Listing 1.9. Evaluation script (all methods, all domains).

```
python RQ2/eval/analyze_rq2_patterns.py
```

Listing 1.10. Pattern analysis script.

7.5 Running the IAA Analysis

```
python iaa/run_iaa_eval.py travel
python iaa/run_iaa_eval.py healthcare
```

Listing 1.11. IAA evaluation.

Output reports are written to `iaa/reports/`.

7.6 Regenerating LLM Outputs (Optional)

Regenerating LLM prediction files from scratch requires an OpenAI API key and incurs API costs.

```
python RQ2/generation/generate_llm_outputs.py \  
  --methods all \  
  --domains all
```

Listing 1.12. LLM generation script.

```
python RQ2/generation/generate_llm_outputs.py \  
  --dry-check \  
  --methods all \  
  --domains all
```

Listing 1.13. Dry-run check (validates configuration without calling the API).